

RAPPORT

User Verbalization Tracker

1. Contexte

Le projet est un prototype web de recherche dont l'objectif est de capturer la parole d'un utilisateur dans un navigateur, la transcrire localement, puis en extraire une représentation structurée :

- texte transcrit avec horodatage par segment et par mot ;
- entités nommées (personnes, lieux, organisations, œuvres, etc.) ;
- relations simples entre ces entités (triplets sujet → prédicat → objet) ;
- alignement de chaque entité avec un graphe de connaissances public (Wikidata, et DBpedia dérivée), de façon à pointer chaque mention vers une fiche canonique non ambiguë.

Le résultat est renvoyé au navigateur et conservé en JSON. La cible n'est pas un produit, mais un *outil d'instrumentation* pour des expérimentations sur le langage parlé : recherche d'information dans les interviews, analyse de podcasts, étude de la verbalisation spontanée. Le code est volontairement court, modulaire et lisible, à la manière d'un prototype académique destiné à être lu, modifié et réutilisé.

Les contraintes principales fixées dès le début :

- **Confidentialité d'abord.** Aucun service tiers de reconnaissance vocale, aucune API commerciale. L'audio et le NLP doivent rester sur la machine. Les seuls appels externes admis sont vers les graphes de connaissances publics, et ils doivent rester optionnels.
- **Simplicité.** Pas d'étape de build côté navigateur, pas de base de données, pas d'orchestration de conteneurs.
- **Lisibilité.** Un fichier = une responsabilité. Une fonction publique par service. Du code que l'on peut lire d'un bout à l'autre.

2. Principes de conception

Quatre principes ont guidé chaque décision, sont développés en section 5 et expliquent les choix qui suivent :

- **Local d'abord, externe en option.** Transcription et NLP locaux. Wikidata appelée par défaut mais désactivable (`ENABLE_ENTITY_LINKING=false`).
- **Composition modulaire.** FastAPI est un *composition root* mince qui appelle quatre services indépendants (`transcribe` → `analyze_text` → `enrich_entities` → `save_result`). Chaque service tient dans un fichier, expose une API publique stable, n'a aucune dépendance au framework web.
- **Dégradation gracieuse.** Tout appel externe est encapsulé dans un helper `_safe()` qui catche les exceptions et renvoie `None` . Le pipeline continue avec moins d'enrichissement.
- **Transparence dans la sortie.** Chaque décision non triviale est exposée dans le JSON renvoyé : pourquoi tel candidat Wikidata a été choisi (`match`), combien de candidats considérés (`candidates`), quel type réel le KG attribue (`kg_type`), si ce type contredit spaCy (`label_mismatch`), avec quelle confiance la langue a été détectée (`language_probability`). Un lecteur peut comprendre *comment* le système est arrivé au résultat.

3. Choix techniques

3.1 Reconnaissance vocale — Whisper (faster-whisper, large-v3-turbo)

Les solutions open source de reconnaissance vocale disponibles aujourd’hui — Whisper, Vosk, Wav2Vec2, Coqui-STT, NVIDIA NeMo — sont globalement **équivalentes en capacité** pour ce cas d’usage : transcription multilingue de qualité correcte, exécution locale, maintenance active, intégration Python disponible. Le choix s’est donc fait sur des critères pratiques, pas sur la qualité brute mesurée :

- **Open source** et poids librement téléchargeables, sans clé API ni compte cloud. C’est l’alignement direct avec la contrainte de confidentialité du brief — pas un argument marketing, mais le seul critère qui *rendait* la contrainte techniquement satisfiable. Toute API commerciale (Google STT, Azure, AWS, AssemblyAI, Deepgram) est exclue d’emblée, indépendamment de sa qualité.
- **Multilingue out-of-the-box** : un seul modèle gère 97 langues, avec détection automatique. Les alternatives comme Vosk ou Coqui exigent un modèle par langue à charger séparément, ce qui complique l’expérience utilisateur dès qu’on veut un sélecteur *Auto-detect / English / Français*.
- **Intégration Python mature** et large écosystème de retours d’expérience publics.

Le prototype a démarré avec la bibliothèque de référence `openai-whisper`. Elle a été remplacée par **faster-whisper** une fois que la montée en qualité a rendu le wrapper de référence inutilisable sur CPU. faster-whisper utilise les mêmes poids mais s’appuie sur le moteur d’inférence CTranslate2 et la quantification int8, ce qui divise le temps d’inférence par environ quatre sur CPU et la consommation RAM par deux. Conséquence concrète : le modèle `large-v3-turbo` — la meilleure qualité disponible sans GPU, quasi-équivalent à `large-v3` mais six fois plus rapide — devient utilisable sur une machine ordinaire. Le détail des paramètres de décodage (beam search, repli de température, VAD Silero) est en §4.1.

3.2 NER et extraction de relations — spaCy

spaCy a été retenu parce qu’il fournit dans un **seul pipeline** préentraîné quatre composants utiles : un tokeniseur, un parseur de dépendances, un étiqueteur morphosyntaxique (POS), et un composant NER. L’extraction de relations par règles s’appuie sur l’arbre de dépendances ; sans parseur, il faudrait ajouter une seconde bibliothèque. Les alternatives purement NER (Flair, pipelines HuggingFace transformers) auraient demandé un assemblage manuel, plus de code, et plus de surface d’erreur.

Modèles `_lg` (large) retenus en production. La progression a été empirique : commencer avec `_sm` puis passer à `_md` puis `_lg`. Les petits modèles produisent des analyses syntaxiques visiblement défectueuses sur des phrases courtes (« Emmanuel Macron dirige la France » étiqueté comme une chaîne `flat:name` continue par `fr_core_news_sm`, zéro relation extraite), ce qui empêche tout travail aval. Les modèles transformer (`_trf`) seraient encore meilleurs mais leur dépendance `curated-tokenizers` ne se compile pas sur Python 3.13 au moment du projet — c’est la voie de montée la plus évidente quand l’environnement le permettra.

L’extraction de relations a été codée **à base de règles** sur les patrons de dépendances. Ce choix est délibéré contre une approche ML (OpenIE, REBEL, transformer fine-tuné) pour trois raisons :

1. **Interprétabilité** : chaque triplet remonte à un patron de dépendances précis qu’un humain peut suivre et déboguer.

2. **Pas de données d'entraînement requises** : on n'a pas de corpus annoté de parole spontanée multilingue.
3. **Coût bas** : parcours en $O(\text{tokens})$ par phrase, pas de modèle supplémentaire à charger.

Le compromis assumé : les règles sont fragiles sur la parole spontanée (faux départs, subordonnées, ellipses), et les prédicats restent des verbes de surface non normalisés à une ontologie.

3.3 Graphes de connaissances — Wikidata + DBpedia dérivée

Wikidata est la **source principale** pour trois raisons précises :

- **Multilingue par QID.** Une fiche est accessible par son identifiant unique (par exemple Q243 pour la Tour Eiffel), quelle que soit la langue du libellé qu'on lui présente. La recherche `wbsearchentities` accepte n'importe quelle langue en `uselang`.
- **Types structurés.** La propriété `P31` (instance of) répond à la question « ceci est de quel type ? » de façon structurée, exploitable comme signal aval. La propriété `P279` (subclass of) permet de remonter la hiérarchie des types quand on en a besoin.
- **API ouverte, pas de clé.** Pas de quota dur, conditions d'usage raisonnables pour un prototype.

L'URI DBpedia est **dérivée du sitelink Wikipédia anglais** du QID Wikidata, plutôt qu'obtenue en interrogeant DBpedia Spotlight comme la spécification initiale le suggérait. Ce changement a été fait après le constat que l'endpoint public de Spotlight renvoyait `HTTP 502 Bad Gateway` pendant le développement. La dérivation déterministe est en fait **strictement plus fiable** que Spotlight :

- Pas d'appel HTTP supplémentaire à DBpedia : le sitelink est déjà inclus dans la réponse `wbgetentities` qu'on fait de toute façon.
- Fonctionne pour les transcriptions non anglaises, parce que Wikidata fait le pont entre les langues via le QID.
- URI canonique : par construction, DBpedia est générée à partir des articles Wikipédia anglais, donc l'URI `https://dbpedia.org/resource/<Titre>` est par définition stable.

Spotlight reste supporté en opt-in (`ENABLE_DBPEDIA_SPOTLIGHT=true`) pour les déploiements qui veulent l'utiliser, en pointant la variable `DBPEDIA_SPOTLIGHT_URL` vers une instance auto-hébergée.

Alternatives écartées : BabelNet (licence académique/commerciale contraignante), YAGO (moins activement maintenu), ConceptNet (orienté common sense plutôt qu'entités), KGs locaux comme Neo4j (hors périmètre pour un MVP).

3.4 Pile web et persistance

- **Backend FastAPI.** Async natif, peu de boilerplate, OpenAPI généré automatiquement, typage Pydantic. Plus léger que Flask + extensions, moins opinionné que Django, et la surface d'API du prototype (un seul endpoint `/analyze`) ne justifie pas plus.
- **Frontend HTML/CSS/JS vanille.** Trois écrans (consentement, enregistreur, résultats) et un appel HTTP. Un framework moderne (React, Vue) serait pure complication ; toute la logique tient en ~300 lignes de JS lisible.
- **Stockage fichier JSON par session.** Une session = un fichier `results/<uuid>.json` avec timestamp UTC. Pas de schéma, pas de migrations, inspectable avec `cat` ou un éditeur, diffable

avec git. La signature `save_result(session_id, data) -> Path` permet de remplacer le stockage par SQLite, PostgreSQL ou un object store en une seule fonction.

4. Améliorations apportées au fil du développement

Cette section retrace l'évolution de chaque brique du pipeline. Le prototype a été itéré une douzaine de fois sur des cas réels, et chaque amélioration a été motivée par un défaut observé en pratique plutôt que par un objectif théorique.

4.1 Transcription

- **v0** : `openai-whisper`, modèle `base`, langue forcée en anglais dans le code. Symptôme observé : un enregistrement en français revenait en anglais, parce que Whisper « traduisait » au lieu de transcrire quand la langue imposée ne correspondait pas à ce qu'il entendait.
- **v1** : la langue choisie dans l'UI est propagée jusqu'à la STT, au NER et à la recherche Wikidata. L'option *Auto-detect* laisse Whisper deviner ; la probabilité de la langue est exposée pour transparence.
- **v2** : beam search (`beam_size=5`, `best_of=5`), repli de température (`0.0 ... 1.0`), `condition_on_previous_text=False`. L'essentiel des répétitions runaway sur clips courts disparaît.
- **v3** : montée du modèle (`base` → `small` → `medium`) avec `openai-whisper`. Qualité visiblement meilleure mais `medium` devient inutilisable en interactif sur CPU.
- **v4** : passage à **faster-whisper**. Mêmes poids, ~4× plus rapide. `large-v3-turbo` devient le défaut : qualité quasi `large-v3` pour 6× moins de temps d'inférence.
- **v5** : VAD Silero activé (`vad_filter=True`, `min_silence_duration_ms=500`). Les silences sont sautés avant l'inférence, ce qui réduit drastiquement les hallucinations. Timestamps au mot exposés.

4.2 NER

- **v0** : `en_core_web_sm`, chargé à l'import. Le français est traité par défaut comme de l'anglais, donc l'étiquetage est très bruité.
- **v1** : `fr_core_news_sm` ajouté, sélection du modèle selon la langue détectée. Les analyses syntaxiques restent défectueuses sur des phrases courtes (« Emmanuel Macron dirige la France » étiqueté en `flat:name`, parse cassé). C'est le signe que la taille du modèle matters plus que la disponibilité.
- **v2** : `_md` (medium). Le parsing devient utilisable, l'étiquetage NER s'améliore sur les prénoms étrangers (« Yasek » étiqueté `PER` au lieu de `LOC`).
- **v3** : `_lg` (large). Modèle actuel. Encore meilleur sur les entités rares ; le coût mémoire et disque reste modeste. Le loader préfère `_lg` et retombe sur `_md` puis `_sm` si nécessaire, ce qui permet à un développeur qui veut juste essayer le projet de commencer avec les plus petits modèles.

4.3 Extraction de relations

- **v0** : SVO naïf, sujet et objet réduits à un seul mot.
- **v1** : groupes nominaux complets en sortie (« Barack Obama », « the United States »), via les *noun chunks* de spaCy ou l'agrégation de `compound` / `flat:name`.
- **v2** : coordinations séparées. « Microsoft and Apple are based in California » produit deux triplets distincts au lieu d'un seul mélange.

- **v3** : phrases copules captées (« Paris is the capital », « Macron est président ») via la dépendance `cop` ou `attr`.
- **v4** : voix passive correctement orientée. L'inversion sujet/objet n'est faite que pour le complément d'agent (« by ... » / « par ... »), pas pour un locatif. « Obama was born in Hawaii » donne `Obama → bear in → Hawaii`, pas l'inverse.
- **v5** : prédicat enrichi de la particule et de la préposition (« work at », « locate in »), filtrage des têtes d'objet à `NOUN/PROPN/PRON/ADJ/NUM/X`.
- **v6** : compatibilité bilingue (étiquettes OntoNotes pour l'anglais, Universal Dependencies pour le français).

4.4 Alignement Wikidata / DBpedia

- **v0** : aucun alignement, juste le NER et les relations.
- **v1** : recherche Wikidata top-1 sur la forme de surface. Marche sur les noms canoniques, résout n'importe quoi sur les homonymes.
- **v2** : 10 candidats récupérés, filtre strict par P31 selon l'étiquette spaCy. C'est la version qui a fait apparaître la découverte la plus instructive du projet : **si spaCy se trompe d'étiquette, le filtre amplifie l'erreur**. Cas réel : le prénom polonais « Yasek » étiqueté `LOC` par `fr_core_news_sm` → l'aligneur filtre la recherche sur des lieux nommés Yasek → trouve un homonyme de lieu. Sans filtre, la recherche aurait remonté un humain en premier.
- **v3** : **sélection rendue agnostique au type spaCy** par défaut. spaCy ne sert plus qu'à détecter les bornes de l'entité ; pour la sélection du candidat, c'est le **type Wikidata réel** (P31) qui fait foi. On prend le premier candidat dont le P31 intersecte une classe enregistrée dans le catalogue. Le type retenu est exposé sous `kg_type`, et un drapeau `label_mismatch` est levé quand le KG contredit spaCy. L'ancien comportement strict reste en opt-in pour les domaines où le NER est fiable.
- **v4** : **désambiguïsation sémantique par contexte de phrase**. Pour les cas ambigus (« Hugo » seul), un modèle multilingue d'embeddings (`paraphrase-multilingual-MiniLM-L12-v2`, ~120 Mo) encode la phrase et chaque description Wikidata. La première version avec seuils permissifs (0.25 / 0.05) réordonnait trop facilement (« Hawaii » → comté, « Microsoft » → Microsoft Research). Les seuils ont été remontés à **0.5 / 0.15**, avec marge doublée quand le top-1 est déjà solide. Le rerank n'intervient que sur signal vraiment fort.
- **v5** : **traversée P279 (subclass of)** sur 2 niveaux. Élargit le vocabulaire de types reconnus sans énumération manuelle (« commune de France » reconnue comme sous-classe d'« entité administrative territoriale »). Résultats cachés en mémoire.
- **v6** : Spotlight conservé en opt-in après le constat d'instabilité ; DBpedia dérivée du sitelink `enwiki` du QID, sans appel supplémentaire.

4.5 Interface

- **v0** : un seul bouton d'enregistrement, pas de consentement, pas de choix de langue.
- **v1** : note de consentement obligatoire (l'utilisateur doit cliquer « I Agree » avant que l'enregistreur s'affiche), sélecteur de langue `Auto-detect / English / Français`, statut et minuteur pendant l'enregistrement.
- **v2** : drag-and-drop d'un fichier audio (mp3, wav, m4a, ogg, webm, flac, jusqu'à 25 Mo) avec bouton « Browse » en secours. La logique d'upload est partagée entre micro et fichier, donc le comportement aval est identique.

- **v3** : suppression d'écho, suppression de bruit et gain automatique activés côté `getUserMedia`. Whisper reçoit un signal notablement plus propre que le défaut navigateur.
- **v4** : affichage du **type Wikidata réel** sur chaque entité (badge jaune avec le `kg_type`), badge rouge quand spaCy et le graphe de connaissances sont en désaccord, badges QID Wikidata (violet, cliquable vers la fiche) et DBpedia (vert, cliquable vers l'URI). Ligne sous la transcription indiquant la langue détectée, sa probabilité et le modèle Whisper utilisé.

5. Réflexions transverses

Quatre principes ont émergé pendant le développement et expliquent beaucoup des choix faits dans le code.

Ne pas propager l'incertitude comme un filtre dur

L'épisode du biais d'étiquette spaCy (§4.4 v2 → v3) en est l'illustration la plus claire. Quand l'étape N d'un pipeline produit un signal **bruité**, l'étape $N+1$ doit le traiter comme un *indice*, pas comme une contrainte. Filtrer durement sur la sortie d'un classifieur peu fiable, c'est garantir que ses erreurs se propagent intactes. La bonne stratégie : laisser passer toutes les hypothèses et trancher en aval avec un autre signal, ou ne filtrer que sur des cas sur-couverts où le faux négatif coûte peu. Dans le projet : agnostique au type par défaut, type spaCy disponible en opt-in.

Surface in, surface out

Un prototype de recherche doit exposer ses propres décisions. La réponse JSON contient plusieurs champs spécifiquement faits pour cela : `wikidata.match` (mécanisme de sélection effectif, par exemple `semantic-rerank(0.62)`), `candidates` (combien considérés), `kg_type` (type réel selon le graphe), `label_mismatch` (désaccord entre spaCy et le KG), `language_probability` (confiance de la détection). Aucun n'est strictement nécessaire pour afficher un résultat ; tous le sont pour le comprendre.

Best-effort partout

Tout appel réseau passe par un helper `_safe()` qui catche les exceptions et renvoie `None`. Panne Wikidata, certificat auto-signé non reconnu, timeout, réponse mal formée : le pipeline continue avec moins d'enrichissement. Cette discipline a permis de gérer l'instabilité connue de l'endpoint DBpedia Spotlight sans renoncer à Spotlight (juste rendu optionnel), et de protéger contre les environnements hostiles (proxy d'entreprise, PATH incomplet).

Configuration > recompilation

Toutes les valeurs ajustables sont des variables d'environnement avec des défauts raisonnables. Un utilisateur peut expérimenter (`WHISPER_MODEL=medium`, `LINKING_USE_SPACY_TYPE=true`) sans toucher au code. Les défauts privilégient la justesse sur la vitesse : un prototype qui se trompe rapidement est pire qu'un prototype qui se trompe lentement.

6. Limites des solutions

Reconnaissance vocale. Même avec `large-v3-turbo` et le VAD, des erreurs subsistent sur les noms propres rares, les accents marqués, les locuteurs non natifs, le jargon de domaine, la parole superposée

et les clips très courts. Pas de **diarisation** (impossible de répondre à « qui a dit quoi ») ni de **streaming temps réel**. Le modèle reste lourd : ~2 Go de RAM, plusieurs secondes par minute d'audio sur CPU.

NER (spaCy `_lg`). Les modèles `_lg` sont CNN-based et ratent encore certaines entités rares ou étrangères. Le modèle **français** n'a que 4 étiquettes (`PER`, `LOC`, `ORG`, `MISC`), bien moins informatif que l'anglais (`PERSON`, `ORG`, `GPE`, `LOC`, `FAC`, `WORK_OF_ART`, `EVENT`, `PRODUCT`, `NORP`, `LANGUAGE`, `LAW`, etc.). Les erreurs de transcription se propagent au NER, et inversement les erreurs du NER biaisent l'alignement aval, même quand on essaie de les contenir (cf. §4.4 v2 → v3).

Extraction de relations (règles). Pas de **résolution de corréférence** (« il », « ça », « he » restent tels quels), pas de prise en compte de la **négation** ni de la **modalité** (un triplet peut être extrait alors que la phrase niait ou hypothétisait), pas de **dépendances longues** ni de subordonnées imbriquées. Les prédicats sont des verbes de surface non normalisés à une ontologie : deux triplets sémantiquement équivalents peuvent être textuellement distincts.

Alignement. La désambiguïsation par similarité de phrase ne s'active que sur des phrases informatives. Seul le **top-N** de la recherche Wikidata est considéré ; un bon candidat au-delà sera raté. Le pipeline dépend de l'accès réseau à Wikidata (dégradation gracieuse si indisponible). **DBpedia** n'est disponible que si la fiche Wikidata possède un article Wikipédia anglais.

Confidentialité. L'audio et le NLP local restent sur la machine. L'alignement envoie le texte des entités à Wikidata (et, si activé, la transcription complète à Spotlight), sauf si `ENABLE_ENTITY_LINKING=false`.

Couche web. CORS ouvert pour le dev local (`allow_origins=["*"]`), à restreindre pour tout déploiement. Limite d'upload de 25 Mo : les longs podcasts doivent être traités via le CLI ou découpés. Pas d'authentification ; le prototype n'est pas conçu pour être exposé tel quel.

7. Références

- Whisper — <https://github.com/openai/whisper> ; faster-whisper — <https://github.com/SYSTRAN/faster-whisper> ; Silero VAD — <https://github.com/snakers4/silero-vad>.
- spaCy — <https://spacy.io>, modèles — <https://spacy.io/models>.
- sentence-transformers — <https://www.sbert.net>.
- FastAPI — <https://fastapi.tiangolo.com>.
- Wikidata — <https://www.wikidata.org> ; propriétés **P31** (instance of) et **P279** (subclass of).
- DBpedia — <https://www.dbpedia.org> ; DBpedia Spotlight — <https://www.dbpedia-spotlight.org>.
- OntoNotes 5.0 — <https://catalog.ldc.upenn.edu/LDC2013T19> ; Universal Dependencies — <https://universaldependencies.org>.
- Schéma IOB2 — Ramshaw & Marcus, *Text Chunking using Transformation-Based Learning*, 1995.
- Paper Whisper — Radford et al., *Robust Speech Recognition via Large-Scale Weak Supervision*, 2022.